

Note: Note: You have the choice to choose any one project THIS ONE or NLP Project

Project Title	RAG-Based Insurance Policy Assistant using Gemini (Gen AI)
Skills take away From This Project	● Retrieval-Augmented Generation (RAG) pipeline implementation ● Integration of Google Gemini LLM for real-time policy Q&A ● Natural Language Processing (NLP) and text similarity search ● Vector database creation and management using FAISS / Pinecone / ChromaDB ● Data preprocessing and document chunking (policy documents, clauses, riders) ● Prompt engineering and contextual query handling ● API integration and chatbot interface creation (e.g., Streamlit or Flask) ● Evaluation of chatbot performance metrics ● Hybrid retrieval (dense + sparse/BM25 search) for improved recall (optional) ● Cross-encoder re-ranking of retrieved chunks for higher precision (optional) ● Agentic RAG: multi-step reasoning and tool-calling (e.g., live claim-status lookup, premium calculator) (optional) ● Guardrails, hallucination detection, and PII redaction for safe, grounded responses (optional) ● Multi-turn conversational memory for context-aware follow-up questions (optional) ● LLM observability and monitoring (LangSmith / Arize Phoenix) for latency, cost, and quality tracking (optional)

Domain	Insurance InsurTech Customer Support Analytics

Problem Statement:

Insurance customers often struggle to understand policy terms, claim procedures, and coverage details buried in lengthy documents. Support teams get overwhelmed with repetitive queries about premiums, exclusions, and claim status. The challenge is to build a smart RAG-based assistant powered by the Gemini LLM that can read policy documents, understand user queries, and generate accurate, plain-language answers grounded in the actual policy text

Business Use Cases:

Policy Clarification – Explains coverage, exclusions, and terms in simple language.

Claims Support – Guides customers through claim filing and status tracking.

Renewal Assistance – Answers premium, renewal date, and add-on queries.

Agent Enablement – Helps insurance agents quickly pull clause-level answers for clients.

Regulatory Compliance – Assists compliance teams in retrieving relevant IRDAI/regulatory clauses..

Approach:

Data Collection: Gather policy wordings, FAQs, claim manuals, and regulatory circulars.

Data Preprocessing: Clean, segment, and chunk long policy documents into retrievable units.

Embedding Generation: Convert chunks into embeddings (e.g., text-embedding-004 or all-MiniLM-L6-v2).

Vector Store Creation: Store embeddings in FAISS, Pinecone, or ChromaDB.

Query Processing: Convert user query into embeddings and retrieve relevant policy clauses.

Response Generation: Feed retrieved context + query into Gemini LLM for a grounded, human-like answer.

Frontend Development: Build a chatbot interface using Streamlit or Flask.

Evaluation: Assess accuracy, relevance, and response latency.

Optional: Hybrid Retrieval & Re-ranking: Combine dense vector search with sparse BM25/keyword search, then re-rank top-k results with a cross-encoder for higher precision.

Agentic Orchestration: Use an agent framework (e.g., LangGraph/LangChain agents) so the assistant can plan multi-step reasoning and call external tools such as a claim-status API or premium calculator.

Guardrails & Safety Layer: Add hallucination detection, PII redaction, and response validation before an answer is returned to the user.

Conversational Memory: Persist session-level context so follow-up questions are answered with awareness of prior turns.

Results:

By the end of the project, learners will be able to:

- Develop a fully functional RAG-based insurance assistant using Gemini LLM.
- Retrieve and apply clause-level policy context dynamically.
- Deploy the assistant for real-world insurance support scenarios.
- Achieve higher accuracy and faster resolution vs. static FAQ documents.

Project Evaluation metrics:

Response Accuracy: Relevance and factual correctness against actual policy text.

Contextual Relevance Score: Quality of retrieved clauses/sections.

Latency: Average retrieval + generation time.

User Satisfaction Score: Simulated user rating.

Precision & Recall (Optional): Retrieval system evaluation.

Faithfulness / Groundedness Score (RAGAS): Measures whether the answer is fully supported by retrieved context.

Hallucination Rate: Percentage of responses containing unsupported or fabricated claims.

Cost per Query: Average token/API cost per user interaction, used to evaluate production feasibility.

Technical Tags:

RAG | LLM | Gemini | Vector Database | NLP | Chatbot |
LangChain | LangGraph | FAISS | Pinecone | Streamlit | Flask |
Prompt Engineering | Text Embedding | Cross-Encoder Re-ranking
| Hybrid Search (BM25) | RAGAS | Guardrails AI | LangSmith |
Docker | Agentic Workflows

Data Set: [kb](#)

- Knowledge Base: Policy documents, FAQs, claim guidelines
- Sample Questions to Test the Bot: e.g., "What is covered under accidental damage?", "How do I file a claim for hospitalization?"
- Source: Insurer FAQs, policy wordings, or open datasets (e.g., insurance claim datasets on Kaggle)
- Format: .csv, .json, or .txt with question-answer pairs or raw policy text
- Variables: **question**, **answer**, **category** (optional — e.g., claims, premium, coverage)

Data Set Explanation:

The dataset contains policy clauses, FAQs, and claim procedures relevant to the insurance domain.

Preprocessing Steps:

- Cleaning and normalization (remove boilerplate legal text, HTML, stopwords)
- Chunking long policy documents into clause-level blocks
- Generating embeddings for each chunk
- Storing them in the vector database for retrieval

Project Guidelines:

Learners should submit:

1. **Source Code Files:** Python scripts or notebooks implementing the RAG pipeline.
2. **Documentation:**
 - Overview of system architecture
 - Model and data explanation
 - Deployment and usage guide
3. **Demonstration App:** Showing chatbot interaction.
4. **Report:** Summary of results, findings, and performance evaluation.

Timeline:

Define the project timeline, including milestones and deadlines.

PROJECT DOUBT CLARIFICATION SESSION (PROJECT AND CLASS DOUBTS)

About Session: The Project Doubt Clarification Session is a helpful resource for resolving questions and concerns about projects and class topics. It provides support in understanding project requirements, addressing code issues, and clarifying class concepts. The session aims to enhance comprehension and provide guidance to overcome challenges effectively.

Note: Book the slot at least before 12:00 Pm on the same day

Timing: Monday-Saturday (4:00PM to 5:00PM)

Booking link : <https://forms.gle/XC553oSbMJ2Gcfug9>

For DE/BADM project/class topic doubt slot clarification session:

Booking link : <https://forms.gle/NtkQ4UV9cBV7Ac3C8>

Session timing:

For DE: 04:00 pm to 5:00 pm every saturday

For BADM 05:00 to 07:00 pm every saturday

LIVE EVALUATION SESSION (CAPSTONE AND FINAL PROJECT)

About Session: The Live Evaluation Session for Capstone and Final Projects allows participants to showcase their projects and receive real-time feedback for improvement. It assesses project quality and provides an opportunity for discussion and evaluation.

Note: This form will Open only on Saturday (after 2 PM) and Sunday on Every Week

Timing:

For BADM and DE

Monday-Saturday (11:30AM to 1:00PM)

For DS and AIML

Monday-Saturday (05:30PM to 07:00PM)

Booking link : <https://forms.gle/1m2Gsro41fLtZurRA>